

UCA BACKLOG · V3 REBUILD

Web Programming Syllabus-Aligned Build & Study Guide

Build window: September 25 – October 10, 2026 · 16 build days

Stack: HTML · CSS · JavaScript · Browser APIs · IndexedDB · Fetch · no React

Primary goal: Rebuild UCA Backlog from an empty folder while deliberately learning the Web Programming syllabus.

Governing rule: You implement. AI teaches, reviews and debugs. Do not copy a complete implementation. The V1 source is a reference for understanding behavior and architecture, not a codebase to paste.

Important: V1 contains a client-side API credential in plaintext. Do not reuse or publish that credential. Rotate/revoke it before using the old deployment, and create a fresh configuration strategy for V2.

This guide follows the same learning loop used by the C-Shell guide: **Study First** → **Build** → **Checkpoint** → **Git** → **Learning note**.

1 · Does the project cover the class syllabus?

The previous UCA guide covered the application features well, but it did not explicitly make every JavaScript topic in your class a learning/build target. V3 fixes that by mapping each syllabus topic to a concrete experiment, function, or application feature.

Class topic	V3 treatment	Where
HTML + browser fundamentals	Semantic structure, forms, inputs, navigation, data-* attributes, DOM relationship	Days 1–2
Viewport + responsive design	Mobile-first layout, media queries, viewport testing	Day 3 + final
CSR	State → render → DOM is explicitly implemented	Day 4
SSR	Learn/document comparison; not implemented because project is frontend-only	Day 4
var / let / const	Small JS scope/hoisting experiments + modern app uses let/const	Day 4
Hoisting + TDZ	Required console experiments before app code	Day 4
Errors	try/catch, validation, network/storage error paths	Days 6, 10, 15
Functions	Declarations, expressions, callbacks, arrows, this, arguments, constructors	Days 4–6
Classes	A small service/model class, including private/static/instance fields	Day 6
Call stack + event loop	Execution-order experiments and async explanation	Day 7
Promises	resolve/reject/then/catch/chaining/Promise.all	Day 7
Fetch	Response, json(), HTTP errors, async requests	Days 8–10
AbortController	Cancel a deliberately slow request / polling request	Day 10
IndexedDB	Personal Notes persistence	Day 11
CRUD + DOM	Weeks/topics/subtopics and notes	Days 5, 9, 11
React	Not included; project is intentionally vanilla JS	N/A

2 · Mental model for V2

Think of the application as five layers, even though it remains a simple frontend project:

Layer	Responsibility	Example functions
HTML	Static semantic structure and accessible controls	form, button, nav, main, textarea
UI / DOM	Create and update visible elements	renderApp(), renderTabs(), renderTopics()
State / logic	Own the current in-memory application state	addTopic(), updateTopic(), deleteTopic()
Services	Talk to browser storage / remote data	fetchSharedData(), saveSharedData(), openNotesDB()
Browser APIs	Actual platform mechanisms	localStorage, fetch, AbortController, IndexedDB

Core rule: UI functions should not know how IndexedDB or the remote API works. The V1 guide already moved toward a service-layer idea; V2 makes that separation explicit.

3 · V1 app.js — how to use it as your reference

The uploaded V1 app.js is 465 lines and gives you a very useful behavioral reference. It already contains state, rendering, local persistence, remote synchronization, presence, CRUD and doubt tracking. Use it after attempting the day's feature yourself: compare your design, identify what V1 solved, then decide what V2 should improve.

Do **not** start by copying functions. First understand the responsibility each function has.

V1 function / handler	What it demonstrates	V2 treatment
updateActivePresence()	Mutating shared presence state	Rebuild as a service/state operation
getActiveUsers()	Filtering stale users with Date.now()	Keep concept; improve data separation
renderActiveUsersHeader()	DOM update from derived state	Rebuild as pure-ish UI rendering
fetchCloudData(force)	fetch + async/await + response.json + state replacement	Split into fetchSharedData() + applySharedData()
queueSaveCloudData()	Debounce + fetch PUT + JSON.stringify	Rebuild as saveSharedData() + debounce wrapper
authBtn.onclick	Event handler + localStorage	Use addEventListener + setUserIdentity()
addWeekBtn.onclick	Validation + state mutation + render + save	Separate addWeek() from UI handler
deleteWeekBtn.onclick	confirm + array/object mutation	Keep behavior; isolate state mutation
renderTabs()	createElement + appendChild + click handlers	Rebuild with event delegation where useful
renderTopics()	Nested data → DOM	Split into smaller render functions
renderSyllabusSummaryDashboard()	Derived dashboard rendering	Keep as a separate view renderer
updateTopicTitle()	Direct state mutation + save	Make updateTopic() a data operation
deleteTopic()	confirm + splice + render + save	Separate confirm/UI from deleteTopic()
addSubtopic()	Nested CRUD	Keep as addSubtopic()
updateSubtopic()	Nested state update	Keep, add validation
deleteSubtopic()	Nested delete	Keep, add validation
toggleDoubt()	Relationship list + toggle semantics	Rebuild as toggleDoubt()
addTopicBtn.onclick	Create object with Date.now()	Move ID creation into createTopic()
render()	Top-level rendering coordinator	Keep as renderApp()
setInterval(...)	Polling + presence + remote fetch	Replace with controlled sync/polling service

V1 behavior worth preserving: Week → Topic → Subtopic hierarchy; doubt participants; active-user presence; summary dashboard; live data synchronization. These are part of what makes the project more than a generic CRUD app.

4 · Recommended V2 function map

These are the functions you should aim to build yourself. You do not need to implement every function on the exact day listed if your architecture naturally changes; the names are a learning checklist, not a command to blindly copy.

Area	Functions / methods to build
App boot	initApp(), bindEvents(), renderApp()
Identity	getStoredUser(), setUserIdentity(), clearUserIdentity(), renderUserStatus()
State	createInitialState(), getCurrentWeek(), setCurrentWeek(), updateState()
Weeks	createWeek(), addWeek(), updateWeek(), deleteWeek(), renderWeeks()
Topics	createTopic(), addTopic(), updateTopic(), deleteTopic(), renderTopics()
Subtopics	createSubtopic(), addSubtopic(), updateSubtopic(), deleteSubtopic()
Doubts	toggleDoubt(), hasUserDoubt(), getDoubtParticipants()
Rendering	renderTabs(), renderTopics(), renderTopicCard(), renderSummary()
Validation	validateWeekName(), validateTopic(), validateSubtopic(), showValidationError()
Local storage	loadIdentity(), saveIdentity()
Remote service	fetchSharedData(), parseSharedResponse(), saveSharedData(), syncSharedData()
Async control	createRequestController(), cancelRequest(), startPolling(), stopPolling()
IndexedDB	openNotesDB(), addNote(), getNote(), updateNote(), deleteNote(), getNotesByTopic()
Notes UI	renderNotes(), openNoteEditor(), handleNoteSubmit(), handleNoteDelete()
Presence	updatePresence(), getActiveUsers(), renderPresence()
Testing helpers	assertStateShape(), logState(), runIntegrationChecks()

DAY 1 — Sep 25

Focus: V1 analysis + architecture · **Level:** HTML/JS foundations

STUDY FIRST

- Read V1 app.js from top to bottom. Trace data → event → mutation → render → save.
- Study: browser document, DOM tree, script execution, scope, objects and arrays.
- Study: V1 state shape and why currentUser/currentWeek/state exist.
- Write your V2 data model on paper before coding.

FUNCTIONS / METHODS TO BUILD

`createInitialState()`, `createWeek()`, `createTopic()`, `createSubtopic()`

CODE / BUILD

- Create empty repo and folders: `css/`, `js/`, `services/`, `assets/`. Add README, .gitignore, LICENSE.
- Create a config.example.js; keep real credentials out of Git.
- Draw the V2 architecture and Week → Topic → Subtopic data relationship.

CHECKPOINT — answer out loud

- Can I explain where one topic lives in memory?
- What is shared vs personal data?
- Which V1 function performs each responsibility?

Git: Make one meaningful commit when the day's stable milestone works. Record one paragraph in LEARNING.md.

DAY 2 — Sep 26

Focus: Semantic HTML skeleton · **Level:** HTML + browser

STUDY FIRST

- Semantic HTML: header/nav/main/aside/section/form/button/input/textarea.
- Forms, labels, data-* attributes, button types, basic accessibility.
- Study viewport meta tag and why responsive pages need it.

FUNCTIONS / METHODS TO BUILD

`initApp()`, `bindEvents()`, `renderUserStatus()`

CODE / BUILD

- Build index.html and notes.html with real semantic elements. Do not create the whole page through JS.
- Create empty containers for weeks, topics, summary and notes.
- Use data attributes for topic/week IDs where useful.

CHECKPOINT — answer out loud

- Why is this an HTML element rather than JS-created markup?
- What does the viewport meta tag do?

Git: Make one meaningful commit when the day's stable milestone works. Record one paragraph in LEARNING.md.

DAY 3 — Sep 27

Focus: CSS + responsive foundation · **Level:** CSS

STUDY FIRST

- Study box model, Flexbox, Grid, media queries, CSS variables, mobile-first design.
- Study viewport widths: desktop/tablet/mobile.
- CSS can be AI-generated, but you must be able to inspect and modify it.

FUNCTIONS / METHODS TO BUILD

No major JS target – wire existing HTML classes cleanly.

CODE / BUILD

- Ask AI to generate the initial CSS only after the HTML structure is stable.
- Build sidebar + main content desktop layout and one-column mobile layout.
- Test at three widths and remove horizontal overflow.

CHECKPOINT — answer out loud

- Can I explain why Grid/Flexbox was used?
- What changes at the mobile breakpoint?

Git: Make one meaningful commit when the day's stable milestone works. Record one paragraph in LEARNING.md.

DAY 4 — Sep 28

Focus: JavaScript core + CSR rendering · **Level:** JS fundamentals + functions

STUDY FIRST

- Study var/let/const, scope, hoisting and TDZ.
- Study function declarations vs expressions, arrow functions, callbacks.
- Study DOM selection, createElement,.textContent, innerHTML and event listeners.
- Study CSR: state → render → DOM. Learn SSR only as a comparison.

FUNCTIONS / METHODS TO BUILD

renderApp(), renderWeeks(), renderTopics(), renderTopicCard(), renderSummary()

CODE / BUILD

- Start with hard-coded JS state. Render Week → Topic → Subtopic into the DOM.
- Change the state object manually and verify the UI changes after renderApp().
- Do tiny console experiments for hoisting and TDZ before application code.

CHECKPOINT — answer out loud

- If state changes, what exact function updates the screen?
- Why does CSR work without a server rendering HTML?

Git: Make one meaningful commit when the day's stable milestone works. Record one paragraph in LEARNING.md.

DAY 5 — Sep 29

Focus: Functions, this, arguments + identity · **Level:** JS functions + localStorage

STUDY FIRST

- Study this in normal functions vs arrow functions.
- Study arguments and why rest parameters are preferred in modern code.
- Study constructors and how object creation differs from factory functions.
- Study localStorage, getItem/setItem, JSON.stringify/parse.

FUNCTIONS / METHODS TO BUILD

getStoredUser(), setUserIdentity(), clearUserIdentity(), renderUserStatus()

CODE / BUILD

- Implement first-visit name flow. No fake default user.
- Persist identity across refresh.
- Make one small standalone constructor experiment and one this/arrow-function experiment; don't force these patterns into the app unnecessarily.

CHECKPOINT — answer out loud

- Why does localStorage work for identity?
- Why is a default fallback user misleading?

Git: Make one meaningful commit when the day's stable milestone works. Record one paragraph in LEARNING.md.

DAY 6 — Sep 30

Focus: Classes + proposal/README · **Level:** Classes + errors

STUDY FIRST

- Study class syntax, constructor, instance properties, private fields (#), static properties/methods.
- Study try/catch and validation errors.
- Understand when a class helps and when a plain object/function is simpler.

FUNCTIONS / METHODS TO BUILD

StorageService(), constructor(), getConfig(), validateWeekName()

CODE / BUILD

- Create a small service class only where it improves separation; e.g. a configuration/storage coordinator.
- Complete README proposal: problem, motivation, goals, features, requirements, architecture, data model, testing, limitations.

CHECKPOINT — answer out loud

- What belongs on the instance?
- What belongs on the class itself?
- What error should the user see vs console?

Git: Make one meaningful commit when the day's stable milestone works. Record one paragraph in LEARNING.md.

DAY 7 — Oct 1

Focus: Call stack, event loop + Promises · **Level:** Async JavaScript

STUDY FIRST

- Study call stack, Web APIs, task/microtask queues and event loop.
- Study Promise states: pending/fulfilled/rejected.
- Build tiny examples using new Promise(resolve,reject), then, catch, finally.
- Study chaining and Promise.all.

FUNCTIONS / METHODS TO BUILD

delay(), loadInOrder(), loadInParallel(), handleAsyncError()

CODE / BUILD

- Do standalone async experiments before touching remote sync.
- Prove the order of synchronous code, Promise callbacks and timers with console logs.
- Use Promise.all for two independent demo tasks.

CHECKPOINT — answer out loud

- What is on the call stack?
- Why doesn't a Promise block the main thread?
- What happens after reject()?

Git: Make one meaningful commit when the day's stable milestone works. Record one paragraph in LEARNING.md.

DAY 8 — Oct 2

Focus: Fetch fundamentals + service layer · **Level:** Fetch API

STUDY FIRST

- Study fetch(), Request/Response, response.ok, response.json().

- Study network failures vs HTTP errors.
- Study `async/await` as Promise syntax.

FUNCTIONS / METHODS TO BUILD

`fetchSharedData()`, `parseSharedResponse()`, `saveSharedData()`, `handleFetchError()`

CODE / BUILD

- Build the remote service independently from UI.
- Fetch sample shared data, validate it, return plain data to the application.
- Do not let render functions call fetch directly.

CHECKPOINT — answer out loud

- What does `fetch()` return?
- Why must `response.json()` be awaited?
- Why check `response.ok`?

Git: Make one meaningful commit when the day's stable milestone works. Record one paragraph in `LEARNING.md`.

DAY 9 — Oct 3

Focus: Syllabus CRUD · **Level:** DOM + events + CRUD

STUDY FIRST

- Study click, submit, input and change events.
- Study event delegation and `preventDefault`.
- Study immutable vs direct mutation trade-offs for simple state.

FUNCTIONS / METHODS TO BUILD

`addWeek()`, `updateWeek()`, `deleteWeek()`, `addTopic()`, `updateTopic()`, `deleteTopic()`, `addSubtopic()`, `updateSubtopic()`, `deleteSubtopic()`

CODE / BUILD

- Implement full Week → Topic → Subtopic CRUD.
- Use `addEventListener` instead of inline `onclick` for V2.
- Validate empty names and confirm destructive deletes.

CHECKPOINT — answer out loud

- For every CRUD operation: what state changed, which function changed it, how did UI update?

Git: Make one meaningful commit when the day's stable milestone works. Record one paragraph in `LEARNING.md`.

DAY 10 — Oct 4

Focus: `AbortController` + synchronization · **Level:** Async requests

STUDY FIRST

- Study `AbortController`, `AbortSignal` and cancellation.
- Study stale data, race conditions and polling.
- Understand why V1 can overwrite state with a stale remote copy.

FUNCTIONS / METHODS TO BUILD

`createRequestController()`, `cancelRequest()`, `syncSharedData()`, `startPolling()`, `stopPolling()`, `updatePresence()`

CODE / BUILD

- Implement controlled remote sync.
- Add a manual Sync button.
- Add active-user presence.

- Use AbortController to cancel a deliberately slow/stale request.
- Use a safe polling interval only after manual sync works.

CHECKPOINT — answer out loud

- What if Browser A and B edit at the same time?
- Which request gets cancelled and why?

Git: Make one meaningful commit when the day's stable milestone works. Record one paragraph in LEARNING.md.

DAY 11 — Oct 5

Focus: IndexedDB personal notes · **Level:** Browser database

STUDY FIRST

- Study IndexedDB: database → object store → record → transaction.
- Learn open/add/get/put/delete and onsuccess/onerror or Promise wrappers.
- Understand why IndexedDB is different from localStorage.

FUNCTIONS / METHODS TO BUILD

openNotesDB(), addNote(), getNote(), updateNote(), deleteNote(), getNotesByTopic()

CODE / BUILD

- Build My Notes per topic.
- Notes must survive refresh and closing/reopening the browser.
- Keep notes private/local and shared syllabus data separate.

CHECKPOINT — answer out loud

- Why is IndexedDB appropriate for notes?
- What is a transaction?
- What happens if the DB fails to open?

Git: Make one meaningful commit when the day's stable milestone works. Record one paragraph in LEARNING.md.

DAY 12 — Oct 6

Focus: Doubts + summary dashboard · **Level:** State relationships

STUDY FIRST

- Study arrays of participants, includes/indexOf, filtering and derived state.
- Study conditional rendering and empty states.

FUNCTIONS / METHODS TO BUILD

toggleDoubt(), hasUserDoubt(), getDoubtParticipants(), renderPresence(), renderSummary()

CODE / BUILD

- Implement 'I have a doubt' toggle.
- Prevent duplicate users.
- Build summary view showing doubt counts and participants.

CHECKPOINT — answer out loud

- If the same student clicks twice, what happens?
- Is the summary stored or derived from state?

Git: Make one meaningful commit when the day's stable milestone works. Record one paragraph in LEARNING.md.

DAY 13 — Oct 7

Focus: Discussion + advanced UI integration · **Level:** Events + application design

STUDY FIRST

- Study event delegation for dynamically-created message controls.
- Study timestamps with Date objects and basic formatting.
- Review callbacks and closures where event handlers capture IDs.

FUNCTIONS / METHODS TO BUILD

`createDiscussion()`, `addMessage()`, `getDiscussion()`, `renderDiscussion()`, `handleMessageSubmit()`

CODE / BUILD

- Add manual Start Discussion flow.
- Add replies, timestamps, members and empty states.
- Do not add voice chat, DMs, notifications or reactions.

CHECKPOINT — answer out loud

- Why should discussion be linked to a topic?
- What happens when there are no active doubts?

Git: Make one meaningful commit when the day's stable milestone works. Record one paragraph in LEARNING.md.

DAY 14 — Oct 8

Focus: BUFFER + bug fixing · **Level:** Review

STUDY FIRST

- No new concept unless required to fix a bug.
- Review the weakest two concepts from Days 4–13.

FUNCTIONS / METHODS TO BUILD

No prescribed new functions. Refactor broken ones.

CODE / BUILD

- Fix CRUD, sync, IndexedDB, mobile and event bugs.
- Compare behavior with V1 and document deliberate V2 changes.
- Run the integration matrix.

CHECKPOINT — answer out loud

- Is the problem conceptual or implementation-related?
- Can I explain the bug before fixing it?

Git: Make one meaningful commit when the day's stable milestone works. Record one paragraph in LEARNING.md.

DAY 15 — Oct 9

Focus: Testing + security + cleanup · **Level:** Full syllabus review

STUDY FIRST

- Review DOM, events, state, functions, classes, promises, fetch, JSON, localStorage, IndexedDB, responsive CSS.
- Review errors, HTTP errors, aborted requests and storage failures.

FUNCTIONS / METHODS TO BUILD

`assertStateShape()`, `logState()`, `runIntegrationChecks()`, `handleError()`

CODE / BUILD

- Test first visit, CRUD, doubts, sync, notes, cancellation, refresh and mobile.
- Search Git history for accidental credentials. Rotate/revoke any V1 credential that was exposed.
- Remove dead code and console noise.

CHECKPOINT — answer out loud

- Can I demonstrate every class topic using this project or a tiny experiment?

Git: Make one meaningful commit when the day's stable milestone works. Record one paragraph in LEARNING.md.

DAY 16 — Oct 10

Focus: Final rehearsal + submission · **Level:** Integration

STUDY FIRST

- No new features. Review only what is needed for explanation.

FUNCTIONS / METHODS TO BUILD

`initApp()`, `renderApp()`, `syncSharedData()`, `openNotesDB()`

CODE / BUILD

- Run six demos from a clean checkout: identity, syllabus CRUD, doubts, sync/presence, IndexedDB notes, responsive UI.
- Explain architecture without opening README.
- Verify README, Git, folder structure and no secrets.

CHECKPOINT — answer out loud

- Can I explain why every important function exists?
- Can I trace one click from event → state → render → persistence?

Git: Make one meaningful commit when the day's stable milestone works. Record one paragraph in LEARNING.md.

5 · V1 → V2 deliberate improvements

- Move API configuration out of app.js. A frontend client cannot truly hide a secret; use restricted public configuration or a backend/proxy if secret protection is required.
- Replace inline onclick/onfocus/onblur handlers with addEventListener and event delegation where appropriate.
- Separate state mutations from rendering and persistence.
- Split the large V1 app.js into modules: app, state, UI, services, storage.
- Do not replace the entire shared state blindly after every fetch; define a synchronization strategy and document its limitations.
- Use AbortController for cancellable requests.
- Use IndexedDB only for personal notes; keep shared class data in its service layer.
- Add explicit loading, empty, error and offline-ish states.
- Add validation and consistent error handling.
- Keep the V1 summary dashboard and doubt/presence behavior because they are useful project-specific features.

6 · V1 reference workflow

When you are stuck, use this exact order:

1. **Try from memory.** Write the function yourself.
2. **Read the V1 equivalent.** For example, if you are implementing topic rendering, inspect V1's renderTopics().
3. **Explain V1.** Identify its inputs, state mutations, DOM operations and side effects.
4. **Close V1.** Rebuild your own version with the V2 architecture.
5. **Compare behavior.** Check what you intentionally changed.
6. **Ask AI only for the missing concept/debugging.**

This is the same principle as your C-Shell guide: the goal is not merely that the program runs; you should be able to explain the mechanism behind it.

7 · Final syllabus coverage checklist

HTML	■ semantic structure ■ forms ■ inputs ■ buttons ■ navigation ■ data-*
Browser	■ DOM ■ viewport ■ responsive design ■ localStorage ■ IndexedDB
Rendering	■ CSR implemented ■ SSR understood/documented
JS variables	■ var ■ let ■ const ■ hoisting ■ TDZ
Errors	■ validation ■ try/catch ■ network/HTTP errors ■ storage errors
Functions	■ declaration ■ expression ■ callback ■ arrow ■ this ■ arguments ■ constructor
Classes	■ class ■ constructor ■ private field ■ static ■ instance
Async	■ call stack ■ event loop ■ Promise states ■ resolve/reject ■ then/catch ■ chaining ■ Promise.all
Fetch	■ fetch ■ Response ■ json() ■ async/await ■ AbortController ■ cancellation
App	■ state ■ render ■ events ■ CRUD ■ sync ■ presence ■ doubts ■ notes

8 · Final project rule

You are rebuilding the project to learn Web Programming, not merely recreating V1. V1 answers 'what behavior did I previously have?'. The syllabus answers 'what must I understand?'. V2 architecture answers 'how can I implement it cleanly?'. Your daily functions connect all three.